ISEL
DEETC
Departamento de Engenharia Electrónica e de Telecomunicações e de Computadores

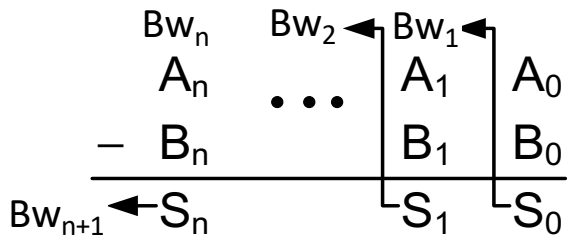
Aula 8

Sumário

- Aritmética binária (continuação)
- Circuitos aritméticos

Subtracção

- Considere a subtracção entre dois números ($A - B$):
  - An,...A1,A0 – diminuendo/aditivo
  - Bn,...B1,B0 – diminuidor/subtrativo



- Ao subtrairmos dois dígitos, existe *Borrow* (Bw_n) se o resultado da subtracção desses dígitos for inferior a zero

Borrow

- Caso $B_n > A_n$ existe a necessidade de adicionar ao algarismo n do diminuendo (aditivo) o valor da base (*borrow*). O valor da base é, de seguida:
  - subtraído ao diminuendo (aditivo) (i.e. $A_{n+1} - 1$)

  Ou
  - adicionado ao diminuidor (subtrativo) (i.e. $B_{n+1} + 1$)

$$\begin{array}{r} 45 \\ -27 \\ \hline 18 \end{array} = \begin{array}{r} 40+5 \\ -20+7 \\ \hline \end{array} = \begin{array}{r} 30+15 \\ -20+\ \ 7 \\ \hline 10+\ \ 8 \end{array} = \begin{array}{r} 40+15 \\ -30+\ \ 7 \\ \hline 10+\ \ 8 \end{array}$$

Impossível
5<7

O mais comum

Subtracção

- Cada um dos dígitos do resultado é obtido através das seguintes expressões:
  - $S_n = |A_n - B_n - Bw_n|_{base}$
  - $Bw_{n+1} = 1$ se $(B_n + Bw_n) > A_n$
- Exemplo para subtracção na base 10 (decimal)
  - $1241_{(10)} - 183_{(10)}$

	A =	1	2	4	1
–	B =		1	8	3
	S =				
		S_3	S_2	S_1	S_0

Subtracção

- Cada um dos dígitos do resultado é obtido através das seguintes expressões:
  - $S_n = |A_n - B_n - Bw_n|_{base}$
  - $Bw_{n+1} = 1$ se $(B_n + Bw_n) > A_n$
- Exemplo para subtracção na base 10 (decimal)
  - $1241_{(10)} - 183_{(10)}$

				Bw_1	
				1	
	A =	1	2	4	11
–	B =		1	8	3
	S =				8
		S_3	S_2	S_1	S_0

Subtracção

- Cada um dos dígitos do resultado é obtido através das seguintes expressões:
  - $S_n = |A_n - B_n - Bw_n|_{base}$
  - $Bw_{n+1} = 1$ se $(B_n + Bw_n) > A_n$
- Exemplo para subtracção na base 10 (decimal)
  - $1241_{(10)} - 183_{(10)}$

			Bw_2	Bw_1	
			1	1	
	A =	1	2	14	1
–	B =		1	8	3
	S =			5	8
		S_3	S_2	S_1	S_0

Subtracção

- Cada um dos dígitos do resultado é obtido através das seguintes expressões:
  - $S_n = |A_n - B_n - Bw_n|_{base}$
  - $Bw_{n+1} = 1$ se $(B_n + Bw_n) > A_n$
- Exemplo para subtracção na base 10 (decimal)
  - $1241_{(10)} - 183_{(10)}$

		Bw_3	Bw_2	Bw_1	
		0	1	1	
	A =	1	02	4	1
–	B =		1	8	3
	S =		0	5	8
		S_3	S_2	S_1	S_0

Subtracção

- Cada um dos dígitos do resultado é obtido através das seguintes expressões:
  - $S_n = |A_n - B_n - Bw_n|_{base}$
  - $Bw_{n+1} = 1$ se $(B_n + Bw_n) > A_n$
- Exemplo para subtracção na base 10 (decimal)
  - $1241_{(10)} - 183_{(10)}$

		Bw_3	Bw_2	Bw_1	
		0	1	1	
	A =	1	2	4	1
–	B =		1	8	3
	S =	1	0	5	8
		S_3	S_2	S_1	S_0

Subtracção

- Cada um dos dígitos do resultado é obtido através das seguintes expressões:
  - $S_n = |A_n - B_n - Bw_n|_{base}$
  - $Bw_{n+1} = 1$ se $(B_n + Bw_n) > A_n$
- Exemplo para subtracção na base 2 (binária)
  - $110110_{(2)}$ - $011101_{(2)}$

	A =	1	1	0	1	1	0
–	B =	0	1	1	1	0	1
	S =						
		S_5	S_4	S_3	S_2	S_1	S_0

Subtracção

- Cada um dos dígitos do resultado é obtido através das seguintes expressões:
  - $S_n = |A_n - B_n - Bw_n|_{base}$
  - $Bw_{n+1} = 1$ se $(B_n + Bw_n) > A_n$
- Exemplo para subtracção na base 2 (binária)
  - $110110_{(2)}$ - $011101_{(2)}$

						Bw_1	
						1	
	A =	1	1	0	1	1	10
–	B =	0	1	1	1	0	1
	S =						1
		S_5	S_4	S_3	S_2	S_1	S_0

Subtracção

- Cada um dos dígitos do resultado é obtido através das seguintes expressões:
  - $S_n = |A_n - B_n - Bw_n|_{base}$
  - $Bw_{n+1} = 1$ se $(B_n + Bw_n) > A_n$
- Exemplo para subtracção na base 2 (binária)
  - $110110_{(2)}$ - $011101_{(2)}$

					Bw_2	Bw_1	
					0	1	
	A =	1	1	0	1	01	0
–	B =	0	1	1	1	0	1
	S =					0	1
		S_5	S_4	S_3	S_2	S_1	S_0

Subtracção

- Cada um dos dígitos do resultado é obtido através das seguintes expressões:
  - $S_n = |A_n - B_n - Bw_n|_{base}$
  - $Bw_{n+1} = 1$ se $(B_n + Bw_n) > A_n$
- Exemplo para subtracção na base 2 (binária)
  - $110110_{(2)}$ - $011101_{(2)}$

				Bw_3	Bw_2	Bw_1	
				0	0	1	
	A =	1	1	0	01	1	0
–	B =	0	1	1	1	0	1
	S =				0	0	1
		S_5	S_4	S_3	S_2	S_1	S_0

Subtracção

- Cada um dos dígitos do resultado é obtido através das seguintes expressões:
  - $S_n = |A_n - B_n - Bw_n|_{base}$
  - $Bw_{n+1} = 1$ se $(B_n + Bw_n) > A_n$
- Exemplo para subtracção na base 2 (binária)
  - $110110_{(2)}$ - $011101_{(2)}$

			Bw_4	Bw_3	Bw_2	Bw_1	
			1	0	0	1	
	A =	1	1	10	1	1	0
–	B =	0	1	1	1	0	1
	S =			1	0	0	1
		S_5	S_4	S_3	S_2	S_1	S_0

Subtracção

- Cada um dos dígitos do resultado é obtido através das seguintes expressões:
  - $S_n = |A_n - B_n - Bw_n|_{base}$
  - $Bw_{n+1} = 1$ se $(B_n + Bw_n) > A_n$
- Exemplo para subtracção na base 2 (binária)
  - $110110_{(2)}$ - $011101_{(2)}$

		Bw_5	Bw_4	Bw_3	Bw_2	Bw_1	
		1	1	0	0	1	
	A =	1	11	0	1	1	0
–	B =	0	1	1	1	0	1
	S =		1	1	0	0	1
		S_5	S_4	S_3	S_2	S_1	S_0

Subtracção

- Cada um dos dígitos do resultado é obtido através das seguintes expressões:
  - $S_n = |A_n - B_n - Bw_n|_{base}$
  - $Bw_{n+1} = 1$ se $(B_n + Bw_n) > A_n$
- Exemplo para subtracção na base 2 (binária)
  - $110110_{(2)}$ - $011101_{(2)}$

		Bw_5	Bw_4	Bw_3	Bw_2	Bw_1	
		1	1	0	0	1	
	A =	1	1	0	1	1	0
–	B =	0	1	1	1	0	1
	S =	0	1	1	0	0	1
		S_5	S_4	S_3	S_2	S_1	S_0

Multiplicação binária

- A multiplicação de M (multiplicando) por m (multiplicador) é dada por:
  - $P = M.m = M.(m_0 + 2.m_1 + \ 4.m_2 + \cdots + 2^{n-1}.m_{n-1})$
  - $P = M.m = M.m_0 + 2.M.m_1 + \cdots + 2^{n-1}.M.m_{n-1}$

			M_3	M_2	M_1	M_0	M
		x		m_2	m_1	m_0	m
			$M_3 . m_0$	$M_2 . m_0$	$M_1 . m_0$	$M_0 . m_0$	
		+ $M_3 . m_1$	$M_2 . m_1$	$M_1 . m_1$	$M_0 . m_1$		
	A_5	A_4	A_3	A_2	P_1	P_0	
+	$M_3 . m_2$	$M_2 . m_2$	$M_1 . m_2$	$M_0 . m_2$			
P_6	P_5	P_4	P_3	P_2	P_1	P_0	P

Multiplicação binária

- A multiplicação de M (multiplicando) por m (multiplicador) é dada por:
  - $P = M.m = M.(m_0 + 2.m_1 + \ 4.m_2 + \cdots + 2^{n-1}.m_{n-1})$
  - $P = M.m = M.m_0 + 2.M.m_1 + \cdots + 2^{n-1}.M.m_{n-1}$

  - Exemplo:
    - $1011_{(2)}$ x $110_{(2)}$

				1	0	1	1	M
			x		1	1	0	m
				0	0	0	0	
		+	1	0	1	1		
			1	0	1	1	0	
	+	1	0	1	1			
1	0	0	0	0	1	0		P

Multiplicação por potências de 2

- A multiplicação por potências de 2 (2^n) é obtida através do acrescento de n 0's à direita
- Exemplos:

Multiplicação por 2 ($10_{(2)}$)

		1	0	1	1	M
	x			1	0	m
		0	0	0	0	
+	1	0	1	1		
	1	0	1	1	0	P

$1011_{(2)} \times 10_{(2)} = 10110_{(2)}$

Multiplicação por 4 = 2^2 ($100_{(2)}$)

			1	0	1	1	M
		x		1	0	0	m
			0	0	0	0	
	+	0	0	0	0		
		0	0	0	0	0	
+	1	0	1	1			
	1	0	1	1	0	0	P

$1011_{(2)} \times 100_{(2)} = 101100_{(2)}$

Circuito somador

Soma binária

- Fazendo a correspondência entre dígitos na base binária e valores lógicos booleanos, podemos extrair as seguintes funções booleanas para S_n e C_{n+1}:

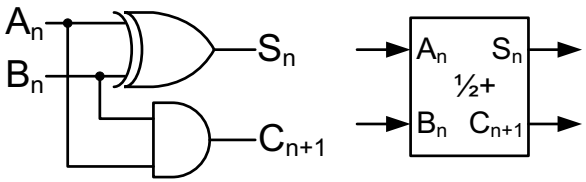
A_n	B_n	S_n	C_{n+1}
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

- $S_n = A_n.\overline{B_n} + \overline{A_n}.B_n = A_n \oplus B_n$
- $C_{n+1} = A_n.B_n$

Semi-somador (*half-adder*)

- O circuito semi-somador (*half-adder*) soma 2 bits de entrada (sem transporte/*Carry*) e produz 1 bit de soma e 1 bit de transporte

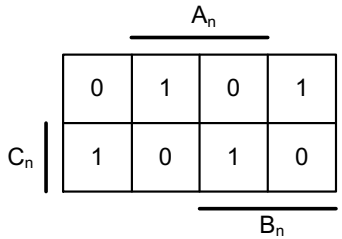
- $S_n = A_n \oplus B_n$
- $C_{n+1} = A_n . B_n$



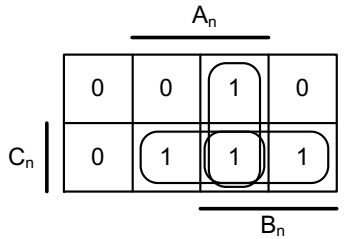
Somador completo (*full-adder*)

- O circuito somador completo (*full-adder*) soma 3 bits de entrada (incluindo o transporte/*Carry* anterior) e produz 1 bit de soma e 1 bit de transporte

A_n	B_n	C_n	S_n	C_{n+1}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1



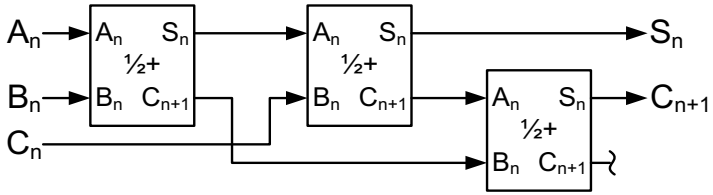
$$S_n = A_n \oplus B_n \oplus C_n$$



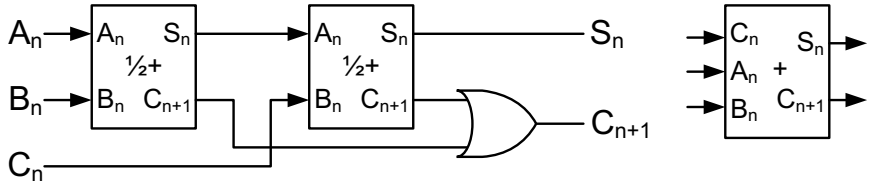
$$C_{n+1} = A_n.B_n + A_n.C_n + B_n.C_n$$

Somador completo (full-adder)

- Podemos construir um somador completo utilizando como bloco base o semi-somador, visto que a soma é associativa: A+B+C = (A+B) + C

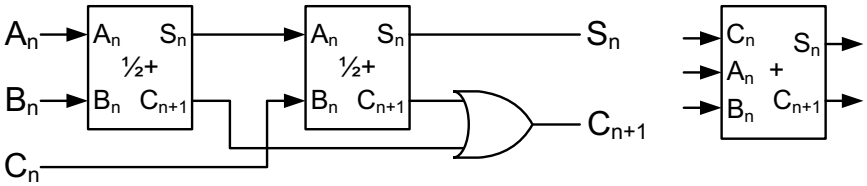


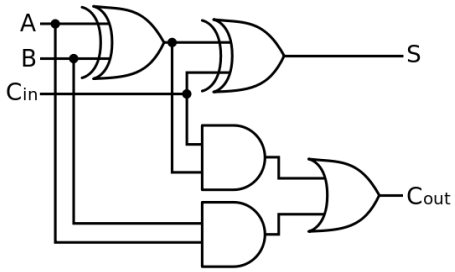
- Visto que o terceiro somador não produz transporte (*Carry*), poderá ser substituído por um XOR ou por um OR



Somador completo (full-adder)

- O circuito somador completo (*full-adder*) soma 3 bits de entrada (incluindo o transporte/*Carry* anterior) e produz 1 bit da soma e 1 bit de transporte

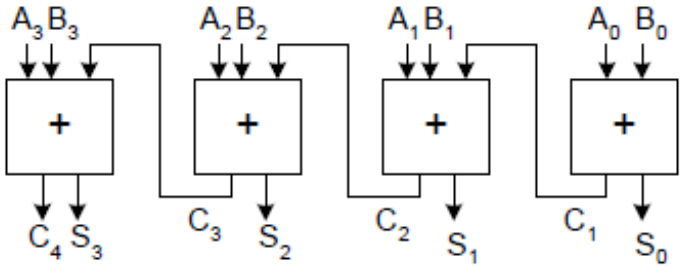




- $S_n = A_n \oplus B_n \oplus C_n$
- $C_{n+1} = A_n \,.\, B_n + (A_n \oplus B_n).\, C_n = A_n.\, B_n + A_n.\, C_n + B_n.\, C_n$

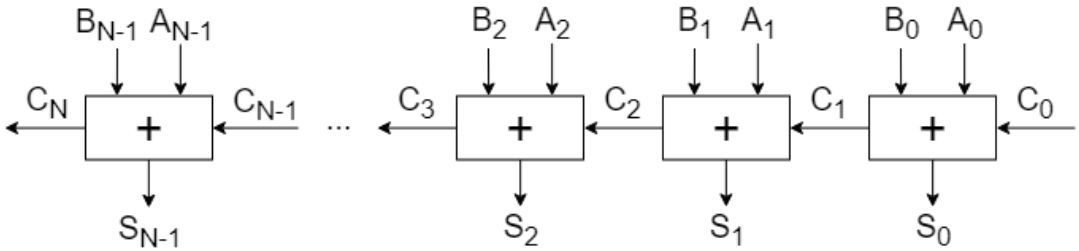
Somador de n bits

- O somador de *n* bits soma dois números (A e B) com *n* bits cada
- As saídas são:
  - Soma ($S_3..S_0$), igualmente com *n* bits
  - Carry (C_n)
- Solução iterativa:
  - Utiliza módulos replicáveis (*full-adders*)



Somador em cascata (*ripple carry adder*)

- Utiliza uma solução iterativa
- Desvantagem:
  - Tempo para o cálculo (i.e. o tempo de propagação) é grande
    - Deve-se à propagação do carry pelos módulos somadores (desde o 1º bit até ao último)
    - No pior caso é N * t_{pfa} (t_{pfa} = tempo propagação do *full adder*, 3 portas lógicas)
- É o somador que requer o menor número de portas lógicas



Outros somadores

- Existem várias alternativas ao somador em cascata, com diversos compromissos entre área e velocidade:
  - *Carry select adder*
  - *Carry look ahead adder*
  - …
- A alternativa mais rápida, mas com maior número de portas lógicas, é o *Carry look ahead*

Temas abordados

Sumário

- Aritmética binária (continuação)
- Circuitos aritméticos

Próxima aula

- Trabalho de laboratório 1 (avaliação)